

Day-2 Operations

KVM Administration, Maintenance & Lifecycle Management

Migration is Day-1. Day-2 is operating KVM in production — patching hosts, managing VM lifecycle, storage maintenance, network changes, and incident response. This guide is the VMware admin's reference for daily KVM operations, with virsh command equivalents for every vSphere operation.

virsh Commands | Host Patching | VM Lifecycle | Storage Mgmt | Snapshot | Live Migration | Incident Response

VMware → virsh Command Reference

Every vSphere operation has a virsh equivalent — most are simpler.

vSphere Operation	virsh Command	Notes
List all VMs	<code>virsh list --all</code>	Shows state: running, shut off, paused
Power on VM	<code>virsh start vm-name</code>	Starts from shut off state
Power off (graceful)	<code>virsh shutdown vm-name</code>	ACPI shutdown signal (clean)
Power off (force)	<code>virsh destroy vm-name</code>	Equivalent to pulling power cable
Reboot	<code>virsh reboot vm-name</code>	ACPI reboot signal
Suspend / Resume	<code>virsh suspend vm</code> / <code>virsh resume vm</code>	Pauses VM in memory
Console (VNC)	<code>virt-viewer vm-name</code>	Or: <code>virsh console vm</code> for serial
Create snapshot	<code>virsh snapshot-create-as vm snap1</code>	Add <code>--disk-only</code> for external snapshot
Revert snapshot	<code>virsh snapshot-revert vm snap1</code>	Reverts to snapshot state
Clone VM	<code>virt-clone --original vm --name vm-clone --auto-clone</code>	Full clone with new MAC and UUID
vMotion (live migrate)	<code>virsh migrate --live vm qemu+ssh://host2/system</code>	Requires shared storage
Edit VM config	<code>virsh edit vm-name</code>	Opens domain XML in \$EDITOR
VM info / specs	<code>virsh dominfo vm-name</code>	CPU, memory, state, UUID
Hot-add CPU	<code>virsh setvcpus vm 8 --live</code>	Requires maxvcpus set in XML
Hot-add memory	<code>virsh setmem vm 16G --live</code>	Requires maxmemory set in XML
Attach disk	<code>virsh attach-disk vm /path/disk.qcow2 vdb</code>	Hot-add while running
Attach network	<code>virsh attach-interface vm bridge br0 --model virtio</code>	Hot-add NIC
Storage usage	<code>virsh domblkinfo vm vda</code>	Allocated vs physical size

Host Patching & Maintenance

Patch KVM hosts with zero VM downtime using live migration.

Rolling Patch Procedure

```
#!/bin/bash
# rolling-patch.sh - Patch KVM host with zero VM downtime
HOST="$1"
DEST_HOST="$2"

echo "=== Patching $HOST (migrating VMs to $DEST_HOST) ==="

# Step 1: Live-migrate all VMs to another host
for vm in $(virsh -c qemu+ssh://$HOST/system list --name); do
    echo "Migrating $vm to $DEST_HOST..."
    virsh -c qemu+ssh://$HOST/system migrate --live --verbose \
        "$vm" qemu+ssh://$DEST_HOST/system
done

# Step 2: Verify no VMs remain
REMAINING=$(virsh -c qemu+ssh://$HOST/system list --name | wc -l)
[ "$REMAINING" -gt 0 ] && echo "ERROR: $REMAINING VMs still on $HOST" && exit 1

# Step 3: Patch the host
ssh $HOST "dnf update -y && reboot"

# Step 4: Wait for host to come back
echo "Waiting for $HOST to reboot..."
while ! ssh -o ConnectTimeout=5 $HOST "uptime" &>/dev/null; do sleep 10; done

# Step 5: Verify host health
ssh $HOST "h2kvmctl doctor"

# Step 6: Migrate VMs back (optional - or leave balanced)
echo "=== $HOST patched successfully ==="
```

Maintenance Schedule

Task	Frequency	Downtime	Command
Security patches	Monthly	0 (live migration)	dnf update --security
Kernel updates	Quarterly	5 min (reboot)	dnf update kernel && reboot

QEMU/libvirt update	Quarterly	VM restart	<code>dnf update qemu-kvm libvirt</code>
Firmware update	Semi-annual	15 min (reboot)	Vendor tool (Dell iDRAC, HPE iLO)
Storage maintenance	Monthly	0	<code>fstrim -av</code> , Ceph scrub
Log rotation	Weekly (auto)	0	logrotate (configured)
Backup verification	Monthly	0	Test restore of random VM

VM Lifecycle Management

Creating, cloning, templating, and decommissioning VMs.

Create New VM

```
# From ISO
virt-install --name new-vm \
  --ram 4096 --vcpus 2 \
  --disk size=50,format=qcow2 \
  --os-variant rocky9 \
  --cdrom /iso/Rocky-9.3-x86_64.iso \
  --network bridge=br0,model=virtio \
  --graphics vnc

# From template (clone)
virt-clone --original template-rocky9 \
  --name prod-app-01 \
  --auto-clone

# From cloud image
wget https://cloud.rocky.page/images/Rocky-9-GenericCloud.qcow2
virt-install --name cloud-vm \
  --ram 2048 --vcpus 2 \
  --import \
  --disk Rocky-9-GenericCloud.qcow2 \
  --cloud-init root-password-generate=on
```

VM Templates

```
# Create template from existing VM
virsh shutdown template-vm
# Generalize (remove machine-specific data)
virt-sysprep -d template-vm \
  --operations defaults,-ssh-userdir
# Mark as template (prevent accidental start)
virsh undefine template-vm --keep-all-storage
# Template disk: /var/lib/libvirt/images/template.qcow2

# Create VM from template (linked clone)
qemu-img create -f qcow2 \
  -b template.qcow2 -F qcow2 \
  new-vm.qcow2

# Define with new XML
virt-install --name new-vm \
  --ram 4096 --vcpus 2 \
  --import --disk new-vm.qcow2 \
  --network bridge=br0,model=virtio
```

VM Decommission

```
# Safe decommission procedure
# 1. Notify stakeholders
# 2. Take final backup
virsh snapshot-create-as vm final-backup
borg create /backup/repo::vm-final \
  /var/lib/libvirt/images/vm.qcow2

# 3. Shutdown and remove
virsh shutdown vm
virsh undefine vm --remove-all-storage

# 4. Clean up DNS, DHCP, monitoring
```

Bulk Operations

```
# Start all VMs
for vm in $(virsh list --name --state-shutoff); do
  virsh start "$vm"
done

# Graceful shutdown all
for vm in $(virsh list --name --state-running); do
  virsh shutdown "$vm"
done

# Snapshot all running VMs
for vm in $(virsh list --name --state-running); do
```

```
# 5. Update CMDB / inventory
# 6. Retain backup for 90 days
```

```
    virsh snapshot-create-as "$vm" \
        "backup-$(date +%Y%m%d)" --disk-only
done

# Report all VM resources
virsh list --all | tail -n+3 | while read _ name state; do
    [ -z "$name" ] && continue
    echo "$name: $(virsh dominfo $name | grep -E 'CPU|Memory')"
```

Storage Operations

Disk management, resize, snapshots, and cleanup.

Common Storage Tasks

Resize VM Disk

```
# Grow QCOW2 disk by 50GB
virsh blockresize vm-name \
    /var/lib/libvirt/images/vm.qcow2 150G

# Or offline:
qemu-img resize vm.qcow2 +50G

# Inside guest: extend filesystem
# Linux (ext4):
growpart /dev/vda 3
resize2fs /dev/vda3

# Linux (XFS):
growpart /dev/vda 3
xfs_growfs /

# Linux (LVM):
pvresize /dev/vda3
lvextend -l +100%FREE /dev/vg/lv
resize2fs /dev/vg/lv
```

Storage Pool Management

```
# List storage pools
virsh pool-list --all

# Create directory-based pool
virsh pool-define-as images dir \
    --target /var/lib/libvirt/images
virsh pool-start images
virsh pool-autostart images

# Check pool usage
virsh pool-info images
# Capacity: 15.00 TB
# Allocation: 4.20 TB
# Available: 10.80 TB

# List volumes in pool
virsh vol-list images

# Clean up orphaned images
# (compare vol-list with VM definitions)
```

Snapshot Management

```
# List snapshots for a VM
virsh snapshot-list vm-name --tree

# Create snapshot with description
virsh snapshot-create-as vm-name "pre-update" --description "Before kernel update"

# Create disk-only snapshot (external, for backup)
virsh snapshot-create-as vm-name "backup" --disk-only --quiesce --atomic

# Revert to snapshot
virsh snapshot-revert vm-name "pre-update"
```



```
# Delete snapshot (merge changes)
virsh snapshot-delete vm-name "pre-update"

# Delete with blockcommit (external snapshots)
virsh blockcommit vm-name vda --active --pivot
```

KVM Day-2 Operations: Simpler Than VMware, More Powerful with Scripting

Every vSphere operation maps to a virsh command that can be scripted, automated, and version-controlled. No GUI dependency, no vCenter single point of failure. Live migration enables zero-downtime patching. Templates and cloud-init enable rapid VM provisioning. The learning curve for VMware admins is 1-2 weeks — most concepts are identical, only the commands change.